

Statistical analysis of the **Upsilon**

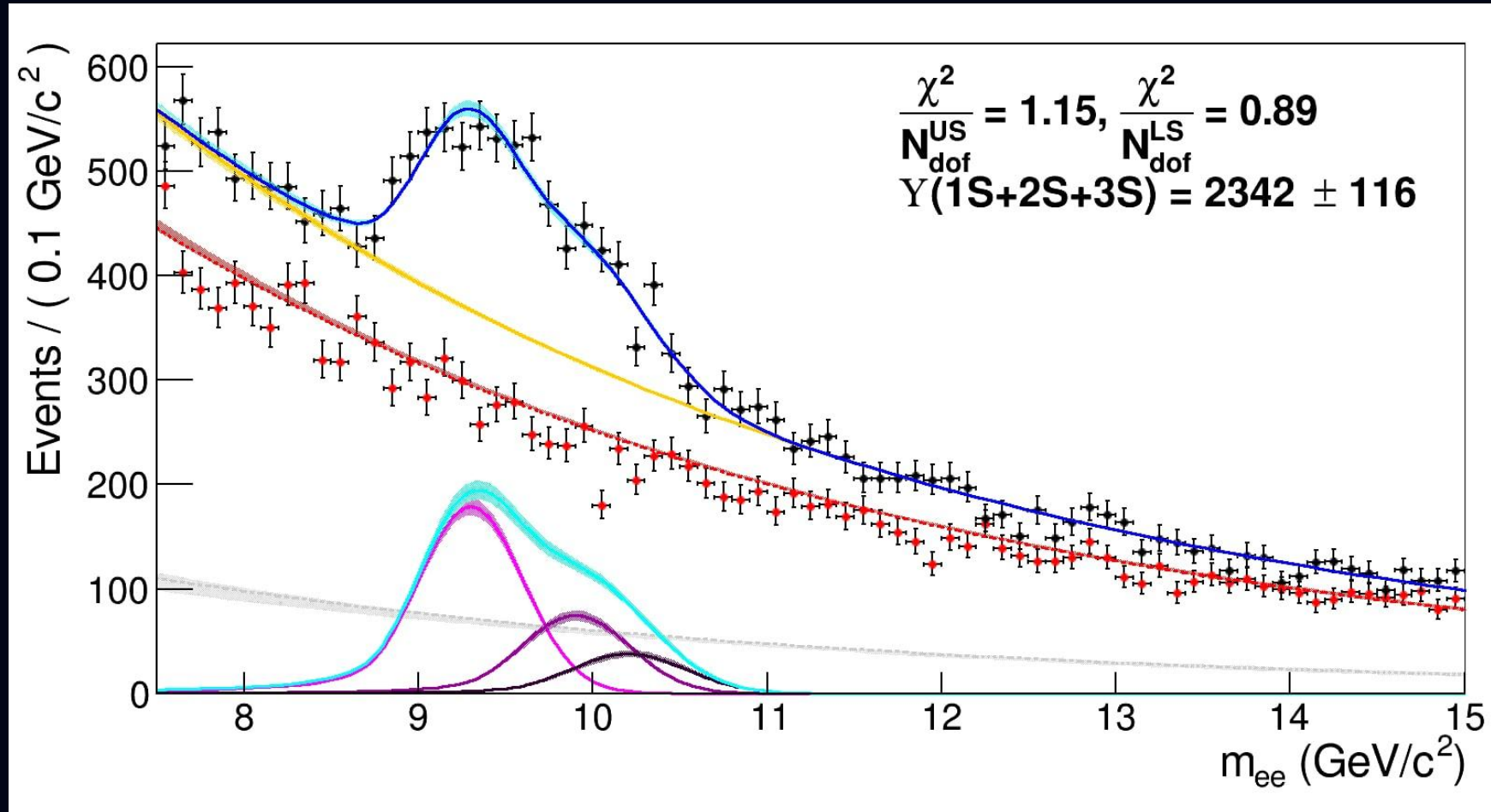
Simultaneous fitting in RooFit

Kristýna Šedová

Upsilon (Υ)



Data from the STAR experiment



Generation of data in PYTHIA

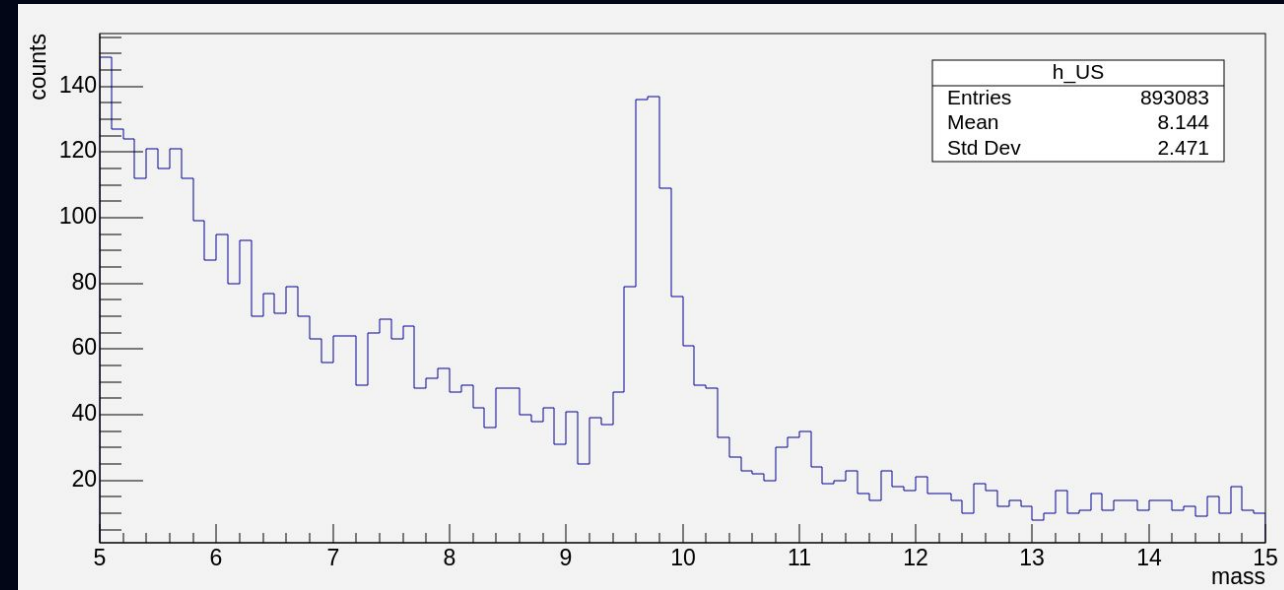
- Simulation p+p collisions

- Processes:

```
Bottomonium:all = on  
WeakSingleBoson:ffbar2gmZ = on  
HardQCD:hardccbar = on  
HardQCD:hardbbbar = on  
SoftQCD:all = on
```

- Invariant mass:

$$m_{ee} = \sqrt{(E_1 + E_2)^2 - |\vec{p}_1 + \vec{p}_2|^2}$$



Source code UpsFitter

- Load file "data.root"
- Run parameters
- Global variables
- Definition of fit_bin function
- Main function

```
// File locations
TString infile = "data.root";

/*
// Run parameters
// Run parameters
*/

Double_t min_mass = 7.6; // x-axis lower limit for fitting
Double_t max_mass = 13.0; // x-axis upper limit for fitting

Double_t bin_width = 0.2; // Bin width for drawing

Int_t sigma = 1; // sigma for error drawing

Bool_t useLinear = true; // Use linear error visualisation (faster, less accurate)

/*
// Global variables
// Global variables
*/

// Data file
TFile * f_data;

// Variables for fitting
RooRealVar mass("mass", "mass", min_mass, max_mass, "GeV/c^{2}");
RooRealVar charge("charge", "charge", -3, 3, "e");

/*
// Function declaration
// Function declaration
*/

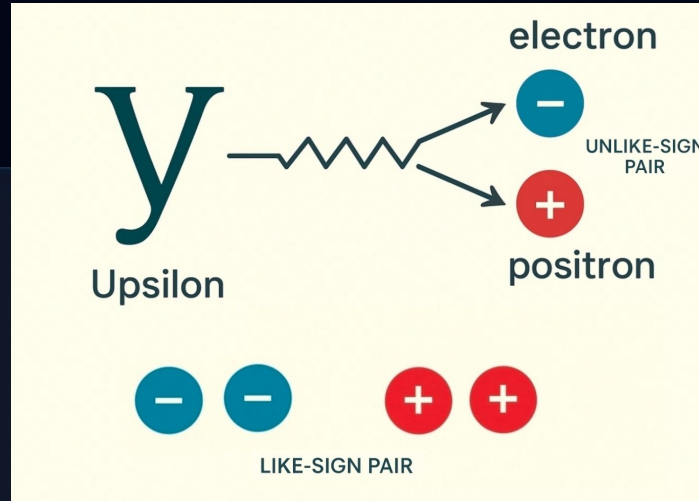
// This function fits the signal via RooFit
// ds_US and ds_LS are the datasets for unlike-sign and like-sign pairs respectively
void fit_bin(RooDataSet * ds_US, RooDataSet * ds_LS);
```

Datasets: US vs LS



Unlike-Sign (US)

Signal + Combinatorial
background
+ Residual background



Like-Sign (LS)

Combinatorial background

TODO 1:

- Open file: UpsFitter.cxx
- For US pairs:

```
RooDataSet * ds_US = new RooDataSet("ds_US", "ds_US", RooArgSet(mass, charge), Import(*tree), Cut("charge==0"));
```

- What will it look like for LS pairs?

TODO 1:

- Open file UpsFitter.cxx
- For US pairs:

```
RooDataSet * ds_US = new RooDataSet("ds_US", "ds_US", RooArgSet(mass, charge), Import(*tree), Cut("charge==0"));
```

- What will it look like for LS pairs?

```
RooDataSet * ds_LS = new RooDataSet("ds_LS", "ds_LS", RooArgSet(mass, charge), Import(*tree), Cut("charge!=0"));
```


Combined dataset

- We assign a US or LS label to each event. This tells RooFit which model to use for that event.
 - We'll need this later
- `RooCategory` = A special type of variable in RooFit that represents discrete categories rather than continuous values
 - For continuous values, the variable `RooRealVar` is used

Mathematical concept

RooFit class

variable

x



RooRealVar

function

$f(x)$



RooAbsReal

PDF

$f(x)$



RooAbsPdf

space point

$\vec{x}$



RooArgSet

integral

$\int_{x_{\min}}^{x_{\max}} f(x) dx$



RooRealIntegral

list of space points



RooAbsData

TODO 2:

- Add the definition to the code :

```
// Define a sample for simultaneous fitting
RooCategory sample("sample", "sample");
sample.defineType("US");
sample.defineType("LS");
```

Used mathematical functions (PDFs)



Exponential

We use this for modeling combinatorial and residual backgrounds



Crystal Ball

Gaussian kernel + power-law tail on the left side. We use this for the signal.



Simultaneous fitting

A combined model that fits both US and LS data simultaneously for maximum statistical accuracy.

TODO 3:

- Use the correct functions to define the PDFs: (fill in the stars placeholder)
 - For background- RooExponential
 - For signal - RooCBShape

TODO 3:

- Use the correct functions to define the PDFs: (fill in the stars placeholder)

- For background- RooExponential
- For signal - RooCBShape

-

- `// Create PDFs`

- `RooExponential *expComb = new RooExponential("expComb", "Comb exp background", mass, bgComb);`

- `RooExponential *expRes = new RooExponential("expRes", "Res exp background", mass, bgRes);`

- `RooCBShape *sig = new RooCBShape("gauss1", "CB signal 1", mass, sig_mu, sig_sigma, sig_alpha, sig_n);`

Fitting parameters

```
RooRealVar bgComb("bgComb","Comb", 0, -1, 1);
```

```
RooRealVar bgRes("bgRes","Res", 0, -1, 1);
```

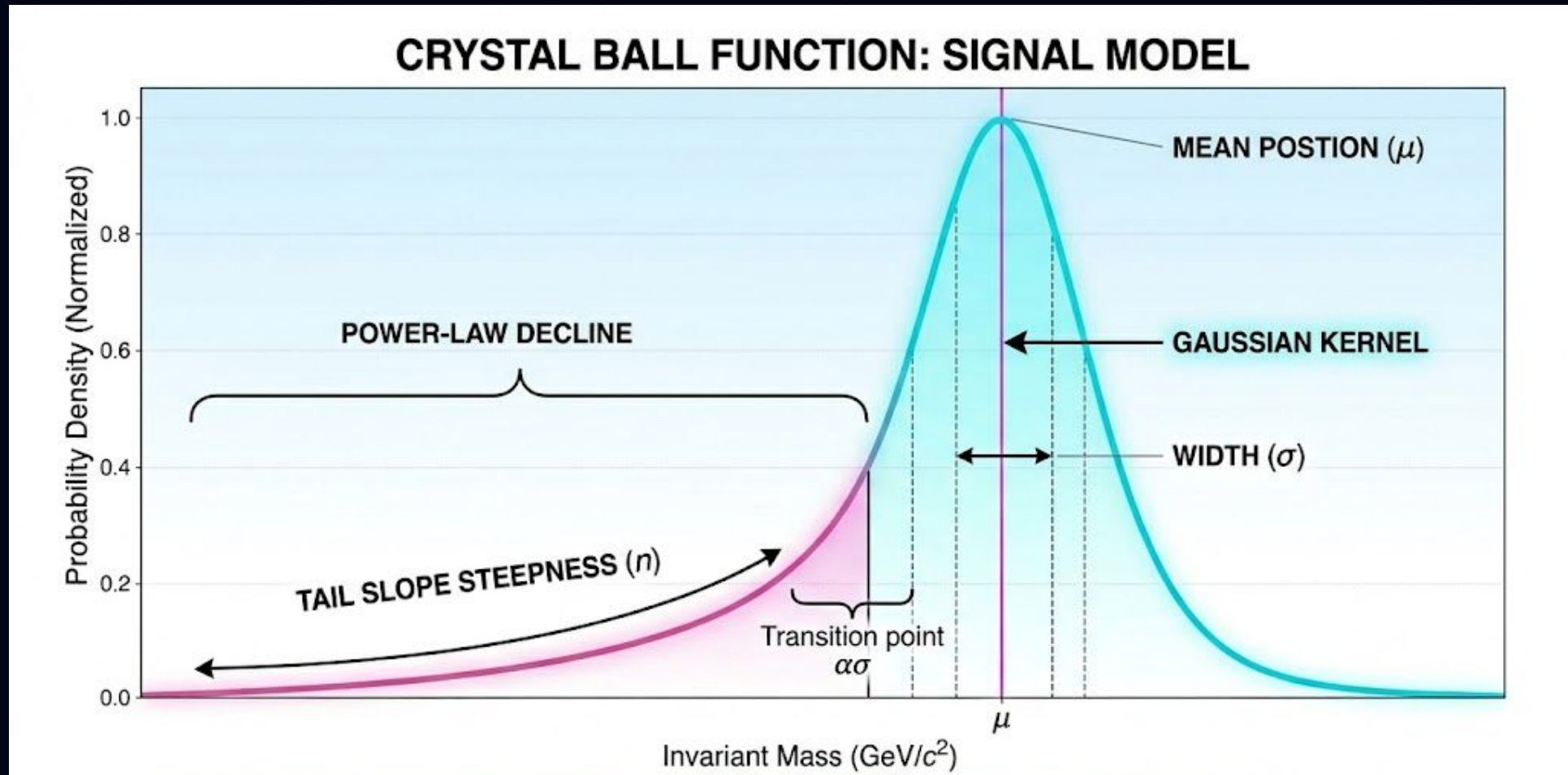
```
RooRealVar sig_mu("sig_mu","#mu", 10, 5, 15);
```

```
RooRealVar sig_sigma("sig_sigma","#sigma", 1, 0, 5);
```

```
RooRealVar sig_alpha("sig_alpha","#alpha", 1, 0, 10);
```

```
RooRealVar sig_n("sig_n","#n", 1, 0, 10);
```

Fitting parameters



Recursive Fraction

Definition and Formula

For a sum of N PDFs ($P_1, P_2, \dots, P_N$) with corresponding parameters $f_1, f_2, \dots, f_{N-1}$ interpreted as recursive fractions, the total PDF is constructed as follows:

$$P_{total} = f_1 P_1 + (1 - f_1) (f_2 P_2 + (1 - f_2) (f_3 P_3 + \dots + (1 - f_{N-1}) P_N))$$

- f_1 is the fraction of P_1 in the total sum.
- f_2 is the fraction of P_2 within the remaining fraction $(1 - f_1)$.
- f_i is the fraction of P_i within the remaining fraction $(1 - f_1)(1 - f_2) \dots (1 - f_{i-1})$

$$P(m) = f_{\text{Comb}} B_{\text{Comb}} + (1 - f_{\text{Comb}}) [f_{\text{Res}} B_{\text{Res}} + (1 - f_{\text{Res}}) S]$$

Recursive Fraction

```
// Define recursive fractions for the components
```

```
RooRealVar bgComb_frac("bgComb_frac","bgComb_frac", 0.5, 0., 1.);
```

```
RooRealVar bgRes_frac("bgRes_frac","bgRes_frac", 0.5, 0., 1.);
```

```
// Ensure the combinatorial background describes the LS data
```

```
bgComb_frac.setVal(ds_LS->sumEntries()/ds_US->sumEntries());
```

```
bgComb_frac.setConstant(true);
```

```
// Add PDFs together with recursive fractions
```

```
RooAddPdf *total_sim = new RooAddPdf("total_sim", "US total",
```

```
RooArgList(*expComb, *expRes, *sig),
```

```
RooArgList(bgComb_frac, bgRes_frac),
```

```
kTRUE);
```

TODO 4:

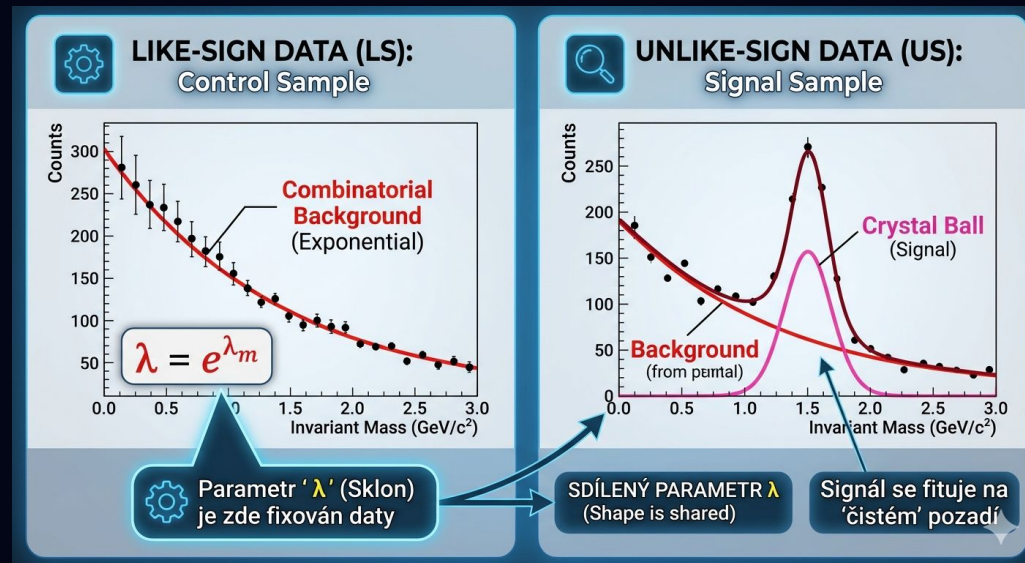
```
// Add PDFs together with recursive fractions
```

```
    RooAddPdf *total_sim = new RooAddPdf("total_sim", "US total",  
RooArgList(*expComb, *expRes, *sig), RooArgList(bgComb_frac, bgRes_frac), kTRUE);
```



Simultaneous fitting

- Simultaneous fitting combines Like-Sign (LS) and Unlike-Sign (US) data.
- Model for LS: only a pure exponential (combinatorial background).
- Shared parameters: the bgComb parameter is shared — it cannot vary independently in the US data; it must simultaneously describe both US and LS data.
- Robustness: it reduces the correlation between signal and background.



TODO 5:

- Code for LS:

```
// Add PDFs to the simultaneous model  
simPdf->addPdf(*expComb, "LS");
```

- What will it look like for US pairs?

TODO 5:

- Code for LS:

```
// Add PDFs to the simultaneous model  
simPdf->addPdf(*expComb, "LS");
```

- What will it look like for US pairs?

```
simPdf->addPdf(*total_sim, "US");
```

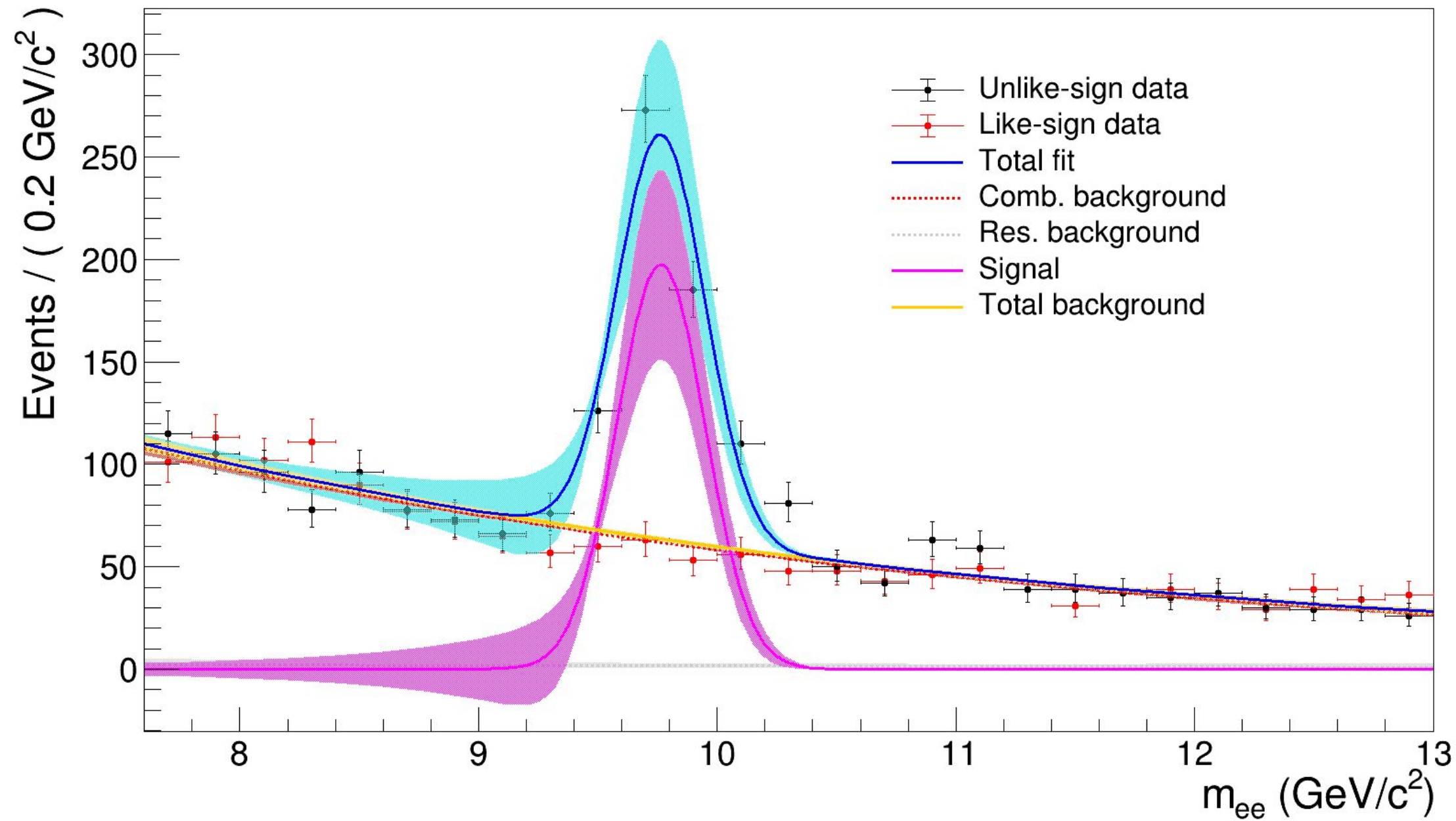
Arguments of fitTo()

- ⚙️ **Minimizer("Minuit2", "migrad"):** It uses the MIGRAD algorithm from the Minuit2 library, which is the standard method for likelihood minimization (the "scan" option is slow).
- 🛡️ **Strategy(2):** It sets the highest minimization level — it performs additional convergence checks
 - 0 - fast but inaccurate
 - 1 - default
 - 2 - High-precision but slow
- 🔍 **RecoverFromUndefinedRegions:** If the minimizer reaches parameter values where the PDF becomes negative or undefined, it tries to move back from that region and continue the fit.

<https://root.cern.ch/doc/master/classRooAbsPdf.html#ab0721374836c343a710f5ff92a326ff5>

TODO 6:

- Run code:
 - `root UpsFitter.cxx`



Output

- US, LS entries
- Yield fraction
- Number of parameters
- Quality of fit
- Significance = $\frac{S}{\sqrt{S+B}}$

Thank you for your attention